**GYANMANJARI INNOVATIVE UNIVERSITY**

Gyanmanjari Institute of Technology – Faculty of Engineering &Technology

# Data Warehouse Design and ETL Implementation

RetailMart: as mall retail data warehouse (star and snow flake schemas)built with Python(Pandas) and SQL

| Student Name | Kothari Tarj Mehulbhai | Enrollment No. | 260205020001 |
|---|---|---|---|
| Subject | DataWarehouseandDataMining (METCE11507) | Semester | 1 (M.Tech) |
| Activity | ALA 1 | Marks | 10 |

## 1. Aim

To design a small **Data Warehouse** for a selected domain (**retail**), identify the fact tables and dimension tables, select an appropriate schema (**star** or **snowflake**), and perform **ETL operations** (extract, clean, transform, load) using Python (Pandas) and SQL scripts. The report contains the schema diagrams, ETL workflow, sample queries and results. *The source data are simulated sample data created for this activity (a fictitious chain, "RetailMart"), with realistic data-quality problems injected so that the cleaning steps can be demonstrated.*

## 2. Domain and Business Requirements

**RetailMart** is a chain of 8 stores in 6 states selling about 36 products in 5 categories (Grocery, Snacks, Electronics, Personal Care, Home). Operational systems record every invoice line. Management needs to analyse historical sales across time, product, store and customer, which the operational (OLTP) systems are not designed for:

| Feature | Operational database (OLTP) | Data warehouse (OLAP) |
|---|---|---|
| Purpose | Run daily business (billing, stock) | Support analysis and decisions |
| Data | Current, detailed, volatile | Historical,integrated,non-volatile (subject-oriented) |
| Design | Normalised (3NF), many small tables | Dimensional(star/snowflake),fewerjoinsfor queries |
| Access | Short read/write transactions | Large read-mostly aggregate queries |

**Business questions to be answered:**

- How do net sales change by year, quarter and month? Is there a festive-season peak?
- Which categories, products and regions contribute most to revenue?
- Which stores and customer segments (age group, gender) are most valuable?
- Which payment modes do customers use? How many different customers visit each store?

**Data warehouse architecture and ETL workflow**

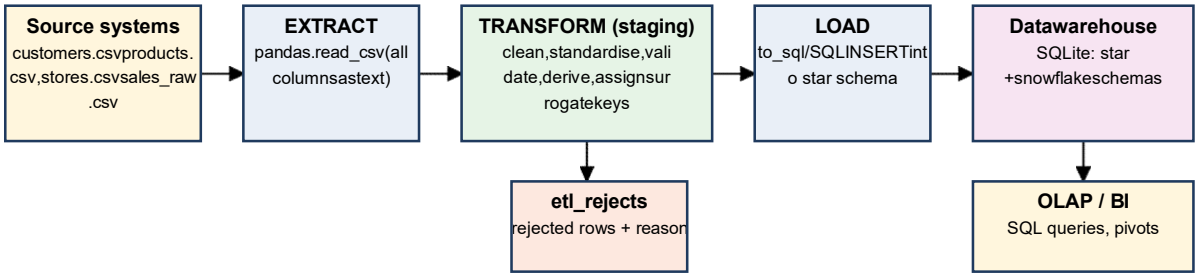



*Figure1:ETLworkflow:sourceextractsarestaged,cleanedandconformed,loadedintothewarehouse,andqueriedby OLAP/BI tools; rejected rows are logged.*

# 3. Dimensional Design

## Grain, facts and dimensions

**Grain** (the level of detail of one fact row): *one line of one sales invoice*. Declaring the grain first ensuresthat all measures and dimensions are consistent.

| Table | Type | Contents / key attributes |
|---|---|---|
| fact_sales | Fact table | Foreignkeystothe5dimensions;degeneratedimensionsinvoice_id, line_no; measures quantity, gross_amount, discount_amount, net_amount, unit_price, discount_pct |
| fact_store_visit | Fact-lessfact table | date_key,customer_key,store_key:records*thatacustomervisiteda store on a date* (an event) without any numeric measure |
| dim_date | Dimension | date_key,date,day,month,quarter,year,weekdayname,is_weekend, season (hierarchy day →month →quarter →year) |
| dim_product | Dimension | product_key,product_id,name,category,subcategory,brand,list_price (hierarchy product →subcategory →category) |
| dim_store | Dimension | store_key, store_id, name, city, state, region (hierarchy store →city → state →region) |
| dim_customer | Dimension | customer_key,customer_id,name,gender,age_group,city,state, signup_year, email_valid; key 0 = 'Unknown / Walk-in' member |
| dim_payment | Dimension (small) | payment_key, payment_mode (UPI, Cash, Card, Wallet) |

## Measure types (additivity)

| Measure | Additivity | Reason / how it is used |
|---|---|---|
| quantity, gross_amount, discount_amount,net_amount | Fully additive | Canbesummedacrossalldimensions(time,product, store, customer) |
| unit_price, discount_pct | Non-additive | Ratios/pricesmustnotbesummed;useweighted averages (e.g. total discount ÷ total gross) |
| (not modelled) stock balance | Semi-additive | Example: inventory on hand can be summed across storesbut*not*acrosstime;itwouldneedaveragingor last-value over time |

## Dimension table and fact table characteristics applied

- **Dimensiontables:**descriptivetextattributes,usedforfilteringandgrouping;onesurrogateintegerkey per row (independent of source keys); contain hierarchies; small and denormalised in the star schema; Type-1 (overwrite) handling of changes.
- **Fact table:** very large and narrow; composed of foreign keys and numeric measures at the declared grain; primary key (invoice_id, line_no); no descriptive text.
- **Fact constellation (galaxy) schema:** the warehouse has two fact tables (fact_sales and fact_store_visit) that share the *conformed* dimensions dim_date, dim_customer and dim_store.

## Star schema

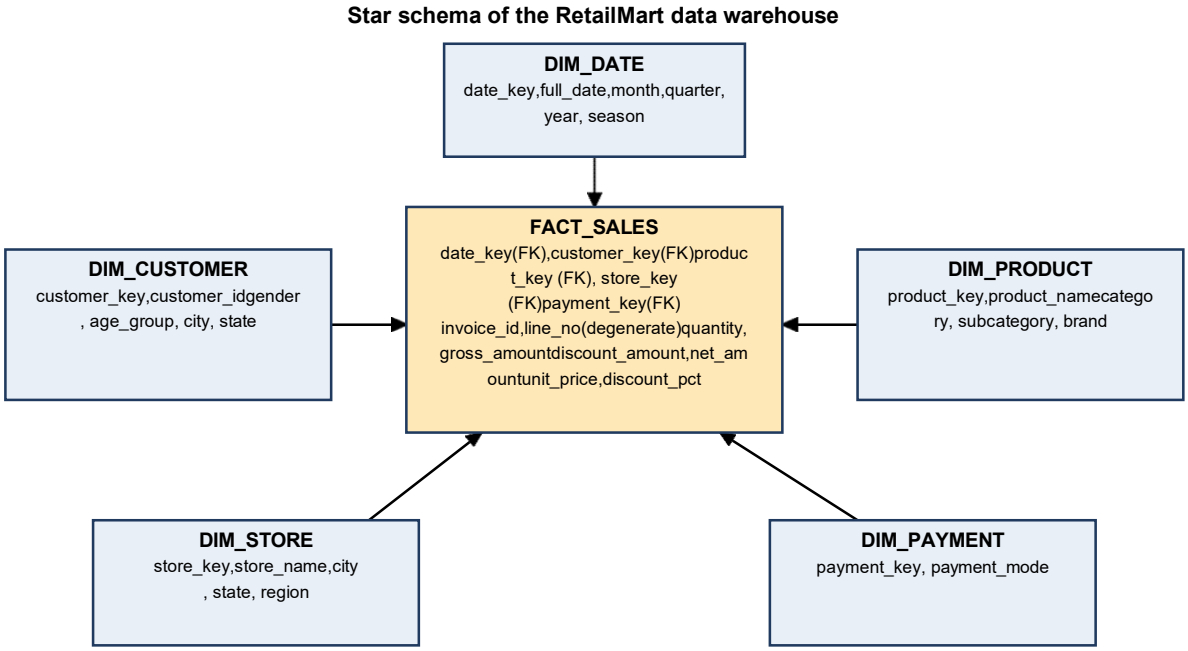



*Figure 2: Star schema: the central fact table is joined directly to denormalised dimension tables.*

## Snowflake schema

Thesnowflakevariantnormalisesthehierarchiesoftheproductandstoredimensionsintoseparatetables. It is also implemented in the database (tables sf_*) so that the two designs can be compared.

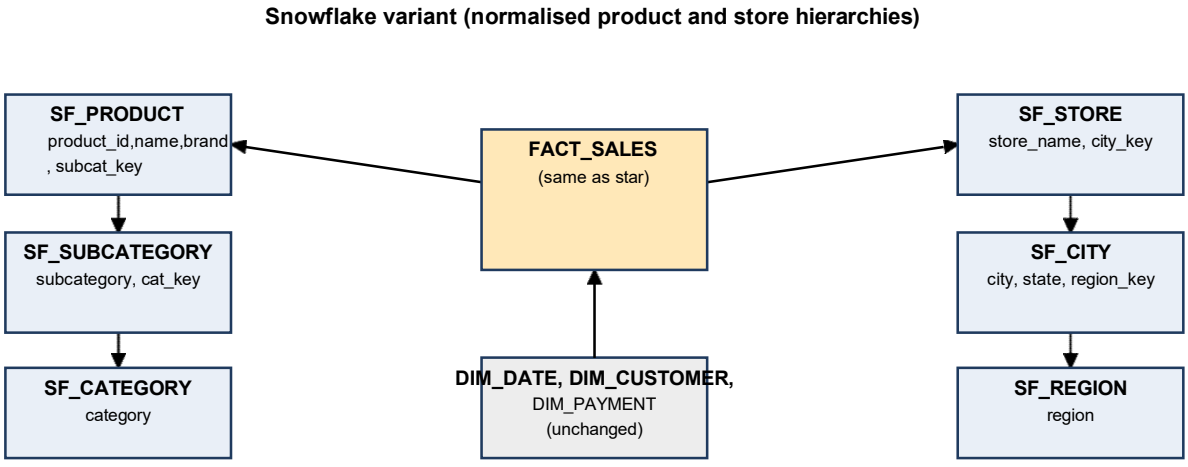



*Figure 3: Snowflake schema: product and store dimensions are split into hierarchy tables.*

| Criterion | Star schema | Snowflake schema |
|---|---|---|
| Joins for ‘sales by category and region’ | 4 joins | 8 joins |
| Measuredquerytime(averageof200 runs) | 2.335 ms | 2.5 ms |
| Result of the query | Total Rs. 4,939,363 | Total Rs. 4,939,363 (identical: True) |
| Redundancy / storage | Somerepeatedtextin dimensions | Lessredundancy,easierupdatesof hierarchy values |
| Understandability | Simpleforbusinessusersand BI tools | More tables and joins to understand |

**Choice:** the **star schema** is selected as the main design because it needs fewer joins, is simpler for analysts and gives equal answers with slightly faster queries; the dimension tables are small, so the storage saved by snowflaking is negligible.

### Table definitions (SQL DDL, excerpt)

```
CREATE TABLE dim_date (date_key INTEGER PRIMARY KEY, full_date TEXT NOT NULL, day INTEGER, month INTEGER, month_nam
                       quarter INTEGER, year INTEGER, day_name TEXT, is_weekend INTEGER, season TEXT);
CREATE TABLE dim_product (product_key INTEGER PRIMARY KEY, product_id TEXT, product_name TEXT, category TEXT,
                          subcategory TEXT, brand TEXT, list_price REAL);
CREATE TABLE fact_sales (                    -- grain: one invoice line
    date_keyINTEGERNOTNULLREFERENCESdim_date(date_key), customer_key
    INTEGER NOT NULL REFERENCES dim_customer(customer_key), product_key
    INTEGER NOT NULL REFERENCES dim_product(product_key), store_key
    INTEGER NOT NULL REFERENCES dim_store(store_key), payment_key
    INTEGER NOT NULL REFERENCES dim_payment(payment_key),
    invoice_id TEXT NOT NULL, line_no INTEGER NOT NULL,        --degeneratedimensions
    quantity INTEGER, gross_amount REAL, discount_amount REAL, net_amount REAL,
    unit_price REAL, discount_pct REAL, PRIMARY KEY (invoice_id, line_no));
CREATETABLEfact_store_visit(date_keyINTEGER,customer_keyINTEGER,store_keyINTEGER,
                                PRIMARYKEY(date_key,customer_key,store_key));        --fact-lessfact
```

## 4. ETL Implementation

The ETL process is implemented in Python (Pandas) with SQL for loading and for building the snowflake tables; the whole script (`etl.py`) is supplied with this report. The source data come from four extracts: `customers.csv` (412rows),`products.csv` (36),`stores.csv` (8)and`sales_raw.csv` (5,385 invoice lines).

| Stage | Operations performed |
|---|---|
| Extract | ReadallfourCSVfileswithPandasastext(dtype=str)sothatnothingisconvertedincorrectly before cleaning. |
| Clean – customers | Remove duplicate customer records; trim spaces and standardise name case; expand state codes(GJ→Gujarat);standardisegender(M/male/F/…→Male/Female);fillmissingcitywith ‘Unknown’; flag invalid e-mail addresses. |
| Clean – sales | Removeexactduplicatelines;parsedateswritteninthreeformats;convertpricesstoredastext (‘Rs. 120’) to numbers; fill missing price from the product master; standardise payment mode (upi/UPI/Upi →UPI); map customer ids that do not exist to the ‘Unknown’ member. |
| Validate(business rules) | Rejectrowswithmissingornon-positivequantity,missingstore,datesinthefuture,ordiscount over 50%. Rejected rows are written to the table etl_rejects with the reason. |
| Transform | Derivegross_amount=quantity×unit_price;discount_amount;net_amount;age_groupfrom birthdate;dateattributes(month,quarter,season,weekend);lookupsurrogatekeysofevery dimension. |
| Load | Create tables (DDL), load the dimensions first and the fact tables afterwards (foreign keys exist),loadthefact-lesstablewithaSELECTDISTINCT,andpopulatethesnowflaketables with INSERT … SELECT statements. Check that no key lookup failed. |

### Core transformation code (excerpt of etl.py)

```
deftransform_sales(s,prod,cust,store):
    s=s.drop_duplicates()                                                    #exactduplicatelines
    s["order_date"] = s["order_date"].apply(parse_date)                      #3dateformats
    s["payment_mode"] = s["payment_mode"].str.strip().str.title().replace({"Upi": "UPI"})
    s["unit_price"] = pd.to_numeric(s["unit_price"].str.replace("Rs.", "", regex=False).str.strip(), errors="coerce
    s["quantity"] = pd.to_numeric(s["quantity"], errors="coerce")
    plist = prod.set_index("product_id")["list_price"].astype(float)
    miss = s["unit_price"].isna()
    s.loc[miss, "unit_price"] = s.loc[miss, "product_id"].map(plist)         #fillpricefrommaster
    orphan = ~s["customer_id"].isin(cust["customer_id"])
    s.loc[orphan, "customer_id"] = "UNKNOWN"                                 # orphan customers -> Unknown member
    rules = [("missing or invalid quantity (<= 0)", s["quantity"].isna() | (s["quantity"] <= 0)),
             ("missingstore",s["store_id"].str.strip()==""),
             ("order date after load date (future)", s["order_date"] > pd.Timestamp("2025-12-31")),
             ("invalid discount (> 50%)", s["discount_pct"] > 50)]
    ...                                                                      #rejectedrows->etl_rejects
    s["gross_amount"] = (s["quantity"] * s["unit_price"]).round(2)
    s["discount_amount"] = (s["gross_amount"] * s["discount_pct"] / 100).round(2)
    s["net_amount"] = (s["gross_amount"] - s["discount_amount"]).round(2)
    returns
```

### ETL results and data quality log

| Step | Rows / count |
|---|---|
| Sales lines extracted | 5,385 |
| Exact duplicate lines removed | 105 |
| Prices stored as text, converted | 155 |
| Missing prices filled from the product master | 42 |
| Orphan customer ids mapped to ‘Unknown’ | 46 |
| Rejected: missing or invalid quantity | 91 |
| Rejected: missing store | 10 |
| Rejected: future order date | 14 |
| Rejected: invalid discount | 12 |
| **Sales lines loaded into fact_sales** | **5,153** |
| Customer records read / duplicates removed | 412 / 12 |
| Missing city filled / invalid e-mail flagged | 19 / 12 |

**Reconciliation:** 5,385 lines extracted −105 duplicates −127 rejected = 5,153 loaded; this equals the 5,153 rows in fact_sales (and 127 rows in etl_rejects), so no data were lost silently. Rows loaded into the warehouse:dim_date731,dim_customer401,dim_product36,dim_store8,dim_payment4,fact_sales 5,153, fact_store_visit 1,499. Total net sales loaded: Rs. 8,613,661; units: 7,986.

## 5. Sample Queries and Results (OLAP Operations)

The data warehouse is queried with SQL on the relational star schema, i.e. a **ROLAP** architecture (a MOLAP server would pre-compute and store the cube in a multidimensional array; HOLAP combines both). The typical OLAP operations are shown below. Amounts are in Indian rupees (net of discounts).

### Total sales by year – Roll-up (to year)

```
SELECT d.year AS year, COUNT(DISTINCT f.invoice_id) AS invoices, SUM(f.quantity) AS units, ROUND(SUM(f.net_amount))
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key
GROUPBYd.yearORDERBYd.year;
```

| year | invoices | units | net sales |
|---|---|---|---|
| 2024 | 675 | 3,636 | 3,674,301 |
| 2025 | 824 | 4,350 | 4,939,360 |

## Sales by year and quarter – Drill-down (year to quarter)

```
SELECT d.year AS year, d.quarter AS quarter, ROUND(SUM(f.net_amount)) AS net_sales
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key
GROUPBYd.year,d.quarterORDERBYd.year,d.quarter;
```

| year | quarter | net sales |
|---|---|---|
| 2024 | 1 | 889,866 |
| 2024 | 2 | 1,109,434 |
| 2024 | 3 | 768,564 |
| 2024 | 4 | 906,438 |
| 2025 | 1 | 1,162,080 |
| 2025 | 2 | 1,207,214 |
| 2025 | 3 | 1,257,197 |
| 2025 | 4 | 1,312,870 |

## Electronics sales by month in Q4 2025 – Drill-down (quarter to month)

```
SELECTd.month_nameASmonth,ROUND(SUM(f.net_amount))ASnet_sales,SUM(f.quantity)ASunits
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key JOIN dim_product p ON p.product_key = f.product_key
WHERE d.year = 2025 AND d.quarter = 4 AND p.category = 'Electronics'
GROUPBYd.month,d.month_nameORDERBYd.month;
```

| month | net sales | units |
|---|---|---|
| Oct | 490,985 | 58 |
| Nov | 284,660 | 35 |
| Dec | 367,070 | 53 |

## Q4 2025 sales by category – Slice (time = Q4 2025)

```
SELECTp.categoryAScategory,ROUND(SUM(f.net_amount))ASnet_sales,SUM(f.quantity)ASunits
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key JOIN dim_product p ON p.product_key = f.product_key
WHERE d.year = 2025 AND d.quarter = 4
GROUPBYp.categoryORDERBYnet_salesDESC;
```

| category | net sales | units |
|---|---|---|
| Electronics | 1,142,715 | 146 |
| Grocery | 102,584 | 642 |
| Personal Care | 33,568 | 199 |
| Home | 21,881 | 92 |
| Snacks | 12,122 | 290 |

## 2025salesofElectronicsandGroceryinWestandSouthregions–Dice (category, region, year)

```
SELECT p.category AS category, s.region AS region, ROUND(SUM(f.net_amount)) AS net_sales, SUM(f.quantity) AS units
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key JOIN dim_product p ON p.product_key = f.product_key
JOIN dim_store s ON s.store_key = f.store_key
WHERE d.year = 2025 AND p.category IN ('Electronics', 'Grocery') AND s.region IN ('West', 'South')
GROUP BY p.category, s.region ORDER BY p.category, s.region;
```

| category | region | net sales | units |
|---|---|---|---|
| Electronics | South | 746,328 | 87 |
| Electronics | West | 2,927,703 | 353 |
| Grocery | South | 70,342 | 424 |
| Grocery | West | 181,507 | 1,115 |

## Data cube: category × region (2025) – Pivot / cube cells

```
SELECTp.categoryAScategory,s.regionASregion,ROUND(SUM(f.net_amount))ASnet_sales
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key JOIN dim_product p ON p.product_key = f.product_key
JOIN dim_store s ON s.store_key = f.store_key
WHEREd.year=2025GROUPBYp.category,s.region;
```

| category | East | North | South | West | Total |
|---|---|---|---|---|---|
| Electronics | 191,375 | 535,328 | 746,328 | 2,927,703 | 4,400,734 |
| Grocery | 12,691 | 45,344 | 70,342 | 181,507 | 309,884 |
| Home | 1,683 | 11,864 | 11,860 | 55,724 | 81,131 |
| Personal Care | 4,443 | 15,357 | 25,574 | 58,221 | 103,595 |
| Snacks | 2,257 | 5,494 | 7,881 | 28,387 | 44,019 |

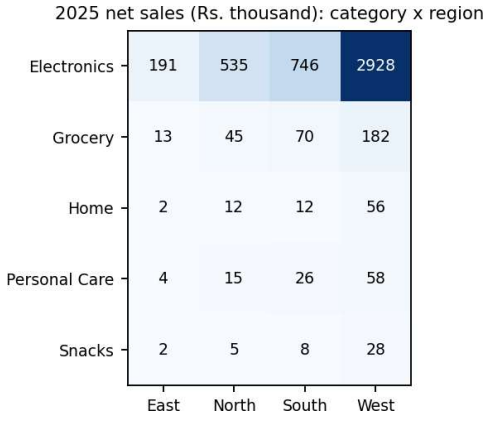



*Figure4:Cubecells(category×region)for2025.Acubeoverthreedimensions(category×region×time)isobtainedby adding the time dimension to the GROUP BY; roll-up removes a dimension or moves up a hierarchy, drill-down does the reverse.*

## Top 5 products by net sales – Ranking

```
SELECT p.product_name AS product, p.category AS category, SUM(f.quantity) AS units, ROUND(SUM(f.net_amount)) AS net
FROM fact_sales f JOIN dim_product p ON p.product_key = f.product_key
GROUPBYp.product_keyORDERBYnet_salesDESCLIMIT5;
```

| product | category | units | net sales |
|---|---|---|---|
| Laptop 14in | Electronics | 125 | 5,422,500 |
| product | category | units | net sales |

| | | | |
|---|---|---|---|
| Smartphone | Electronics | 120 | 1,733,250 |
| Basmati Rice 5kg | Grocery | 347 | 207,173 |
| Wheat Flour 5kg | Grocery | 338 | 94,787 |
| Power Bank 10000mAh | Electronics | 56 | 81,000 |

## Net sales by store – Roll-up (to store)

```
SELECT s.store_name AS store, s.region AS region, ROUND(SUM(f.net_amount)) AS net_sales, COUNT(DISTINCT f.invoice_i
FROM fact_sales f JOIN dim_store s ON s.store_key = f.store_key
GROUPBYs.store_keyORDERBYnet_salesDESC;
```

| store | region | net sales | invoices |
|---|---|---|---|
| RetailMartAhmedabad Mall | West | 1,367,637 | 235 |
| RetailMartMumbai Andheri | West | 1,363,478 | 249 |
| RetailMart Surat City | West | 1,310,192 | 215 |
| RetailMartChennaiT Nagar | South | 1,242,820 | 147 |
| RetailMartBhavnagar Central | West | 1,176,302 | 228 |
| RetailMartDelhi Connaught | North | 1,107,519 | 189 |
| RetailMartBengaluru Indiranagar | South | 581,965 | 158 |
| RetailMartKolkataPark Street | East | 463,748 | 78 |

## Averagedailysales:festiveseasonvsregularseason–Slicebyseason attribute

```
SELECT d.season AS season, ROUND(SUM(f.net_amount) / COUNT(DISTINCT d.date_key)) AS avg_daily_net_sales
FROM fact_sales f JOIN dim_date d ON d.date_key = f.date_key
GROUPBYd.season;
```

| season | avg daily net sales |
|---|---|
| Festive (Oct-Nov) | 13,978 |
| Regular | 13,553 |

## Customersegments(knowncustomers):agegroup–Roll-up(customer attribute)

```
SELECT c.age_group AS age_group, COUNT(DISTINCT c.customer_key) AS customers, ROUND(SUM(f.net_amount)) AS net_sales
FROM fact_sales f JOIN dim_customer c ON c.customer_key = f.customer_key
WHEREc.customer_key<>0GROUPBYc.age_groupORDERBYnet_salesDESC;
```

| age group | customers | net sales |
|---|---|---|
| 51+ | 145 | 2,936,268 |
| 36-50 | 118 | 2,739,136 |
| 26-35 | 85 | 1,811,226 |
| **age group** | **customers** | **net sales** |
| 18-25 | 48 | 1,070,229 |

### Payment mode share of net sales – Roll-up (payment)

```
SELECTy.payment_modeASpayment_mode,ROUND(SUM(f.net_amount))ASnet_sales,
       ROUND(100.0 * SUM(f.net_amount) / (SELECT SUM(net_amount) FROM fact_sales), 1) AS share_pct
FROM fact_sales f JOIN dim_payment y ON y.payment_key = f.payment_key
GROUPBYy.payment_keyORDERBYnet_salesDESC;
```

| payment mode | net sales | share pct |
|---|---|---|
| UPI | 3,968,270 | 46.1 |
| Cash | 2,074,869 | 24.1 |
| Card | 1,727,413 | 20.1 |
| Wallet | 843,110 | 9.8 |

### Distinctcustomersvisitingeachstore(fact-lessfacttable)–Fact-lessfact query

```
SELECT s.store_name AS store, COUNT(*) AS customer_visits, COUNT(DISTINCT v.customer_key) AS distinct_customers
FROM fact_store_visit v JOIN dim_store s ON s.store_key = v.store_key
GROUPBYs.store_keyORDERBYcustomer_visitsDESC;
```

| store | customer visits | distinct customers |
|---|---|---|
| RetailMart Mumbai Andheri | 249 | 192 |
| RetailMart Ahmedabad Mall | 235 | 181 |
| RetailMart Bhavnagar Central | 228 | 179 |
| RetailMart Surat City | 215 | 175 |
| RetailMart Delhi Connaught | 189 | 152 |
| RetailMart Bengaluru Indiranagar | 158 | 124 |
| RetailMart Chennai T Nagar | 147 | 125 |
| RetailMart Kolkata Park Street | 78 | 73 |

### OLAP operations demonstrated

| Operation | Meaning | Example in this report |
|---|---|---|
| Roll-up | Aggregatetoahigherlevelofahierarchyor drop a dimension | 5.1 month/quarter →year; store →region |
| Drill-down | Move to a more detailed level of a hierarchy | 5.2 year →quarter; 5.3 quarter →month |
| Slice | Select one value of one dimension | 5.4 time = Q4 2025 |
| Dice | Selectvaluesontwoormoredimensions (sub-cube) | 5.5 category ∈{Electronics, Grocery}, region ∈ {West, South}, year = 2025 |
| Pivot (rotate) | Rotate axes to see another view | 5.6 category × region cross-tab |

## 6. Observations from the Warehouse

*These observations describe the simulated data and illustrate how a warehouse supports decisions.*

- Net sales grew by about **34%** from 2024 (Rs. 3,674,301) to 2025 (Rs. 4,939,360).

- Electronics(laptopsandsmartphones)generate**87%**ofQ42025revenuealthoughtheyaccountforfar fewer units than Grocery; the West region contributes about 67% of Electronics sales.
- Laptops and smartphones are the two best-selling products by revenue; groceries lead by volume.
- UPI is the most used payment mode (46% of sales), followed by cash and cards.
- Customers aged 36 and above account for the largest share of sales by known customers.

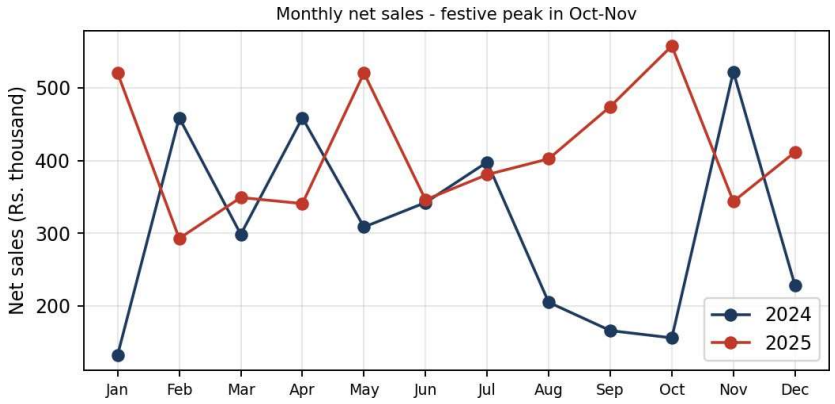



*Figure 5: Monthly net sales for 2024 and 2025.*

## 7. Conclusion

A dimensional data warehouse for a retail domain was designed with one sales fact table, one fact-less fact table and five conformed dimensions; star and snowflake versions were built and found to give identical results, and the star schema was selected. An ETL pipeline in Python and SQL cleaned the raw extracts (duplicates, inconsistent formats, missing values, orphan keys, invalid values), logged every rejected row, reconciled row counts and loaded the warehouse. OLAP operations and sample queries answered the business questions. **Possible improvements:** Type-2 slowly changing dimensions for customer city, an inventory snapshot fact table (semi-additive), incremental (delta) loading and a MOLAP cube tool.

## References


- JiaweiHan,MichelineKamberandJianPei,*DataMining:ConceptsandTechniques*,Morgan Kaufmann Publishers.
- Alex Berson and Stephen J. Smith, *Data Warehousing, Data Mining and OLAP*, Tata McGraw-Hill.
- Prabhu, *Data Warehousing: Concepts, Techniques, Products and Applications*, Prentice Hall of India.
- Ralph Kimball and Margy Ross, *The Data Warehouse Toolkit*, 3rd ed., Wiley (dimensional modelling).